Structural Stability Diagnostics for Pseudo-Random Number Generators

Seed Sensitivity, Functional Variance Dispersion, and Cross-Architecture Empirical Analysis

Pascal Montmory
ENINCA Consulting
hello@eninca.com

Version 1.0 – May 2026

Abstract

Classical statistical test batteries such as TestU01 BigCrush and PractRand are necessary but insufficient to assess the reliability of pseudo-random number generators (PRNGs) in Monte Carlo simulations. Generators compatible with these tests can still exhibit seed-dependent variations in functional variance, leading to unstable estimator behaviour across runs.

This paper introduces a structural diagnostic framework that complements traditional statistical tests by evaluating three properties: seed sensitivity, functional stability across representative integrands (identity, sinusoidal, quadratic, and rare-event), and bit-exact reproducibility across heterogeneous architectures (ARM64 CPU, Metal GPU, and CUDA GPU).

Using `Montmory_CTACM` as a controlled proprietary test vehicle, the reported results show that common PRNGs can display inter-seed dispersion ratios up to approximately 25× on practical Monte Carlo integrands, despite passing full BigCrush. Imported BigCrush logs for seed 42 report 160/160 passed statistics for the test vehicle on CPU ARM, Metal GPU, and CUDA GPU backends.

These results suggest that seed-conditioned Monte Carlo stability should be treated as a first-class empirical criterion in PRNG evaluation for large-scale scientific and industrial applications. The current artifact is computational evidence, not a mathematical proof of generator quality, and identifies the additional artifacts required for independent verification.

Keywords: PRNG, Monte Carlo, seed sensitivity, functional stability, cross-architecture reproducibility, statistical diagnostics.

Resume en francais

Cette note technique presente un cadre de diagnostic structurel pour l'evaluation des generateurs pseudo-aleatoires dans les simulations Monte Carlo. L'objectif n'est pas de remplacer les batteries statistiques classiques, comme TestU01 ou PractRand, mais de les completer par des mesures de sensibilite a la seed, de dispersion de variance fonctionnelle et de reproductibilite inter-backend.

Le generateur proprietaire `Montmory_CTACM` est utilise uniquement comme vehicule experimental controle. Sa recurrence interne, ses constantes et son code d'implementation ne sont pas publies dans cette note. La contribution principale porte sur la methode d'evaluation et sur le protocole empirique.

Les logs BigCrush importes indiquent 160/160 statistiques passees pour la seed 42 sur CPU ARM, Metal GPU et CUDA GPU. Les diagnostics structurels rapportes montrent que

des generateurs passant les batteries classiques peuvent neanmoins presenter une dispersion inter-seed importante de la variance Monte Carlo, notamment sur les evenements rares.

Le document doit etre lu comme une evidence computationnelle et methodologique, pas comme une preuve mathematique de qualite, de periode, de securite ou d'equidistribution.

Contents

1 Introduction

Monte Carlo simulation depends critically on pseudo-random number generators. In practice, the quality of a PRNG is often assessed through statistical batteries such as TestU01 [1] and PractRand [2]. These tools are essential: they detect many classes of distributional defects in long output streams and remain a necessary condition for any serious empirical PRNG claim.

However, practitioners frequently encounter questions that are not fully answered by a single-stream statistical battery:

- Does the Monte Carlo variance of a practical estimator depend strongly on the seed?
- Do different integrands reveal different seed-dependent behaviour?
- Can the same generator be replayed bit-exactly across CPU and GPU backends?
- Can a generator pass BigCrush while still showing functional instability in a Monte Carlo workload?

This note proposes a structural diagnostic layer intended to answer these questions. The term *structural stability* is used here in a practical Monte Carlo sense: low sensitivity of functional variance to the seed, and robustness of this behaviour across computational backends. It is unrelated to the dynamical-systems meaning of structural stability.

2 Related Work and Positioning

Classical PRNG evaluation has several complementary components. The first component is theoretical: period, equidistribution, lattice structure, recurrence properties, and independence properties under explicit assumptions. These results are generator-specific and require disclosure of the transition rule and output map. The present note does not provide such theorems for `Montmory_CTACM`, because the generator is not disclosed.

The second component is empirical distributional testing. TestU01, introduced by L'Ecuyer and Simard [1], and tools such as PractRand [2], are designed to identify departures from expected random behaviour over long finite streams. Their role is central. A generator that fails strong statistical batteries requires careful justification before being used in demanding Monte Carlo workloads.

The third component is the construction of streams suitable for parallel computation. Counter-based generators such as Random123 and Philox [6] address the need for reproducible parallel streams and substreams. Combined multiple recursive generators such as MRG32k3a [7] also provide well-studied stream and substream structures. PCG [4], xoshiro-family generators [5], and Mersenne Twister variants [3] represent different trade-offs between state size, speed, equidistribution properties, and implementation simplicity.

The component emphasized here is workload-facing structural behaviour. In this view, the object of interest is not only the stream considered in isolation, but the effect of seed, integrand, backend, and finite-window replay on quantities used by practitioners. This is why the proposed framework focuses on empirical variance of Monte Carlo functionals and on reproducibility across backends.

The closest existing concerns are stream reproducibility, substream independence, and variance robustness in simulation. The present framework is narrower and more operational: for a declared seed set, finite horizon, replicate count, integrand family, and backend set, it records whether Monte Carlo variance is stable. It does not replace the mathematical study of periods, lattice structure, or substream construction. It also does not replace TestU01 or PractRand. It adds a conditional finite-window diagnostic layer that can be attached to practical simulation reports.

Evaluation layer	Typical question	Evidence type
Mathematical analysis	What can be proved from the recurrence and output map?	Theorem under explicit hypotheses
Statistical battery	Does one or more streams show detectable defects?	Empirical pass/fail and p-value behaviour
Structural diagnostics	Is Monte Carlo performance stable under seeds, functionals, and backends?	Conditional finite-window variance and replay evidence

Table 1: Positioning of structural diagnostics relative to existing PRNG evaluation layers.

3 Scope and Non-Claims

The note deliberately separates method, evidence, and claims.

Method. The diagnostic framework is public: definitions, observables, seed perturbations, functional variance ratios, and backend reproducibility checks are stated explicitly.

Test vehicle. The proprietary generator `Montmory_CTACM` is used as a controlled test vehicle. It is treated as a black-box deterministic PRNG with a fixed public interface:

```
init(seed) -> state_0
next_u64(state_t) -> (state_{t+1}, word_t)
uniform01(word_t) -> u_t in [0, 1)
```

Non-claims. This document does not disclose the generator recurrence, prove its period, prove equidistribution, prove cryptographic security, or claim that TestU01 success implies functional stability. Statistical evidence is labelled as computational evidence unless a mathematical proof is provided.

4 Formal PRNG Model

Let a PRNG be represented by the tuple

$$G = (\mathcal{S}, \mathcal{P}, \mathcal{A}, T, O, U),$$

where:

- $\mathcal{S}$ is the state space;
- $\mathcal{P}$ is the parameter space;
- $\mathcal{A}$ is the output alphabet, typically fixed-width integer words;
- $T_p : \mathcal{S} \to \mathcal{S}$ is the transition rule for parameter p;
- $O_p : \mathcal{S} \to \mathcal{A}$ is the output map;
- $U : \mathcal{A} \to [0, 1)$ maps output words to floating-point uniforms.

Given parameter p, seed s, and horizon n, the finite output window is

$$X_n(p, s) = (x_0, \ldots, x_{n-1}) \in \mathcal{A}^n,$$

and the corresponding uniform sequence is

$$U_n(p, s) = (u_0, \ldots, u_{n-1}) \in [0, 1)^n.$$

A reproducible empirical claim must specify:

- the seed model and any seed derivation rule;
- the parameter set p;
- the output map and floating-point conversion convention;
- the horizon n;
- the observable applied to the finite output window;
- the command, code version, and expected output or hash.

5 Structural Diagnostic Framework

5.1 Finite-Window Observables

Let Φ be a declared observable applied to either integer words or uniforms:

$$\Phi : \mathcal{A}^n \to \mathbb{R}^d \quad \text{or} \quad \Phi : [0,1)^n \to \mathbb{R}^d.$$

Examples include bit-frequency statistics, transition matrices, TestU01 scores, Monte Carlo estimates, or stream hashes. The notation $\Phi(G(p,s,n))$ is shorthand for applying Φ to the finite window generated by G under (p,s,n).

5.2 Seed Sensitivity

Let B be a finite set of tested seeds. A seed perturbation rule is a map

$$N_\delta : \text{Seed} \to \mathcal{P}(\text{Seed}),$$

where $N_\delta(s)$ is the declared neighbourhood of seed s. It may be defined through one-bit flips, Hamming balls, arithmetic offsets, cryptographic derivations, or any reproducible finite rule.

For an observable Φ, local seed sensitivity can be measured by

$$\Delta_\Phi(s) = \max_{s' \in N_\delta(s)} \left\| \Phi(G(p,s,n)) - \Phi(G(p,s',n)) \right\|.$$

5.3 Monte Carlo Functional Variance

For an integrand $f : [0,1) \to \mathbb{R}$, seed s, sample size N, and replicate r, define the Monte Carlo estimate

$$I_{s,r}(f) = \frac{1}{N} \sum_{t=1}^{N} f(u_{s,r,t}).$$

For R independent replicates under the same seed model, define

$$\bar{I}_s(f) = \frac{1}{R} \sum_{r=1}^{R} I_{s,r}(f)$$

and the seed-conditioned empirical variance

$$V_s(f) = \frac{1}{R-1} \sum_{r=1}^{R} \left(I_{s,r}(f) - \bar{I}_s(f) \right)^2.$$

The inter-seed dispersion ratio is

$$\rho_f(B) = \frac{\max_{s \in B} V_s(f)}{\min_{s \in B,\, V_s(f) > 0} V_s(f)}.$$

A large $\rho_f(B)$ indicates that Monte Carlo variance depends materially on the seed, even if the generator passes a distributional battery.

5.4 Diagnostic Integrands

The reported protocol uses the following simple but informative integrands:

$$f_1(u) = u, \qquad f_2(u) = \sin(2\pi u), \qquad f_3(u) = u^2, \qquad f_4(u) = \mathbf{1}_{\{u>0.99\}}.$$

The rare-event indicator is especially useful because defects that are not visible in first-order averages may appear more clearly in tail-sensitive functionals.

5.5 Open Structural Score

The proprietary internal scoring functions are not disclosed. A public non-proprietary score can nevertheless be defined from the published observables.

Let $\mathcal{F}$ be the diagnostic integrand set. For each $f \in \mathcal{F}$, define

$$Z_s(f) = \frac{\log V_s(f) - \mathrm{median}_{b\in B} \log V_b(f)}{\mathrm{MAD}_{b\in B}(\log V_b(f))},$$

where MAD is the median absolute deviation, with a declared positive floor if needed.

An open structural seed score is

$$S_{\mathrm{open}}(s) = \frac{1}{|\mathcal{F}|} \sum_{f\in\mathcal{F}} |Z_s(f)|.$$

This score is not claimed to be unique. Its role is to make representative seed selection reproducible without revealing proprietary metrics.

5.6 GSQI: Generator Structural Quality Index

For public reporting, one may aggregate several dispersion ratios into a bounded index called the Generator Structural Quality Index:

$$\mathrm{GSQI}(G; B, \mathcal{F}) = \exp\left(-\frac{1}{|\mathcal{F}|} \sum_{f\in\mathcal{F}} \log \rho_f(B) \right).$$

Under this convention, $\mathrm{GSQI} \in (0, 1]$ when all $\rho_f \geq 1$. Values closer to 1 indicate lower average inter-seed dispersion over the declared integrand family. This is a diagnostic score, not a proof of randomness or security.

5.7 Transition Instability $\epsilon(T)$

Let M_s be a transition matrix estimated from a symbolic projection of the output stream generated from seed s. For example, uniforms may be discretized into q classes and transitions between successive classes counted.

Define the seed-averaged transition matrix

$$\bar{M} = \frac{1}{|B|} \sum_{s\in B} M_s.$$

A finite-window transition instability can be defined as

$$\epsilon(T; B, n, q) = \max_{s\in B} \left\| M_s - \bar{M} \right\|_\infty.$$

This quantity measures backend- and seed-dependent deviations in a declared finite symbolic transition model. It should not be confused with an asymptotic theorem about the generator transition function.

6 Cross-Backend Reproducibility

For a declared backend set $\mathcal{B}$, bit-exact single-lane reproducibility means

$$\forall b_1, b_2 \in \mathcal{B},\ \forall t < n : \quad x_t^{(b_1)} = x_t^{(b_2)}.$$

A practical repository check stores

$$H_b(s, n) = \mathrm{SHA256}(x_0^{(b)} \| x_1^{(b)} \| \cdots \| x_{n-1}^{(b)})$$

and requires

$$\forall b_1, b_2 \in \mathcal{B} : \quad H_{b_1}(s, n) = H_{b_2}(s, n).$$

The current repository defines this requirement, but does not yet include raw stream hashes proving bit-exact equality. Imported BigCrush logs support statistical evidence on three backends, not a direct proof of stream equality.

7 Experimental Methodology

7.1 Generators Compared

The reported comparison includes:

- `Montmory_CTACM`, used as a proprietary controlled test vehicle;
- MT19937 [3];
- MT19937-64;
- PCG32 [4];
- xoshiro256** [5];
- Philox4x32-10 [6];
- MRG32k3a [7].

7.2 Monte Carlo Protocol

The structural-dispersion protocol is:

- 100 tested seeds per generator;
- $N = 10^6$ samples per seed;
- $R = 8$ replicates;
- four diagnostic integrands: identity, sinusoidal, quadratic, and rare event;
- representative seeds selected from the 100-seed profile.

The reported tables use five representative structural seeds per generator for compact exposition. The complete proprietary sensitivity metrics are not disclosed.

7.3 TestU01 Protocol

The imported repository artifacts currently cover BigCrush logs for seed 42 on:

- CPU ARM on Darwin;
- Metal GPU on Darwin;
- CUDA GPU on Linux.

SmallCrush logs are reported but not yet imported. An x86 CPU raw log is also not yet imported.

Backend	Suite	Seed	Statistics	Result
CPU ARM, Darwin	BigCrush	42	160	all passed
Metal GPU, Darwin	BigCrush	42	160	all passed
CUDA GPU, Linux	BigCrush	42	160	all passed

Table 2: Imported BigCrush logs for `Montmory_CTACM`.

8 Results

8.1 Imported BigCrush Logs

Table 2 summarizes the imported BigCrush artifacts.

The imported logs report TestU01 version 1.2.3 and end with `All tests were passed`. The SHA-256 hashes are listed in Appendix A.

8.2 Extracted Log Summary

Backend	Generator label	CPU time	Parsed p-value range
CPU ARM	`PRNG_MONTMORY_CPU_ARM_CTAM`	02:08:48.69	0.02–0.9985
Metal GPU	`PRNG_MONTMORY_GPU_METAL_CTAM`	02:16:17.77	2.6e-3–0.9994
CUDA GPU	`PRNG_MONTMORY_GPU_CUDA_CTAM`	02:37:03.42	2.6e-3–0.9994

Table 3: Extracted metadata from the imported BigCrush logs.

The number of parsed `p-value of test` lines is 254 in each log, larger than the 160 BigCrush statistics because several tests report multiple p-values.

8.3 HAL Artifact Status

The HAL upload should be a single autonomous PDF. The Git repository is not the HAL artifact; it is supporting material. Table 4 states which parts of the current public material are already covered by this document and which parts remain future work.

Item	Current status in this PDF	Remaining gap
Diagnostic framework	Included with formulas for ρ_f, GSQI, and $\epsilon(T)$	Final peer-review refinements
BigCrush results	Included for CPU ARM, Metal, CUDA	SmallCrush and x86 logs absent
Inter-seed dispersion	Included as reported table	Raw per-seed table absent
Cross-backend reproducibility	Formal requirement included	Stream hashes absent
Performance versus Philox	Not included as throughput result	Benchmark table and commands absent
Proprietary generator details	Explicitly excluded	Requires separate disclosure model if ever needed

Table 4: Current HAL-readiness status of the document.

8.4 Inter-Seed Dispersion

Table 5 reports the structural dispersion values currently documented in the repository. These values are reported empirical results; the raw per-seed tables are still pending import.

PRNG	Backend	min V_{u^2}	max V_{u^2}	ρ_{u^2}	min V_R	max V_R	ρ_R
`Montmory_CTACM`	CPU	2.0×10^{-7}	2.1×10^{-6}	~ 10	1.1×10^{-6}	3.8×10^{-6}	~ 3.4
`Montmory_CTACM`	GPU lane 1	2.0×10^{-7}	2.1×10^{-6}	~ 10	1.1×10^{-6}	3.8×10^{-6}	~ 3.4
MT19937	CPU	1.3×10^{-7}	1.3×10^{-6}	~ 10	2.5×10^{-7}	3.5×10^{-6}	~ 14
MT19937-64	CPU	3.4×10^{-7}	6.6×10^{-7}	~ 1.9	2.0×10^{-7}	4.9×10^{-6}	~ 25
PCG32	CPU	1.4×10^{-7}	2.1×10^{-6}	~ 15	4.7×10^{-7}	1.5×10^{-6}	~ 3.2
xoshiro256**	CPU	2.4×10^{-7}	1.4×10^{-6}	~ 5.9	3.3×10^{-7}	3.5×10^{-6}	~ 10.5
Philox4x32-10	CPU	2.4×10^{-7}	1.1×10^{-6}	~ 4.6	3.1×10^{-7}	3.5×10^{-6}	~ 11
MRG32k3a	CPU	1.9×10^{-7}	1.5×10^{-6}	~ 7.7	4.2×10^{-7}	1.6×10^{-6}	~ 3.9

Table 5: Reported inter-seed dispersion on quadratic and rare-event integrands. Here V_R denotes the empirical variance for $R(u) = \mathbf{1}_{\{u>0.99\}}$. Protocol: $N = 10^6$ samples, 8 replicates, 100 tested seeds, compact table over representative structural seeds.

The table reports extrema and ratios, but not confidence intervals. The raw 100-seed replicate table is not yet included in the public repository, so error bars would be inappropriate in this version. A future reproducibility bundle should include the full per-seed table, replicate-level estimates, and bootstrap or asymptotic intervals for each ρ_f.

9 Discussion

The reported results support a methodological point: passing BigCrush and exhibiting low functional variance dispersion are distinct properties. BigCrush evaluates distributional behaviour under a large collection of tests. The structural diagnostics proposed here evaluate the stability of Monte Carlo behaviour under controlled changes in seed, integrand, and backend.

The rare-event integrand is particularly informative in the reported data. Several common generators show larger dispersion on $\mathbf{1}_{\{u>0.99\}}$ than on smoother functionals. This does not imply that these generators are defective in a general sense. It indicates that functional variance can depend on the seed in ways that are not directly summarized by pass/fail battery results.

The proposed framework is therefore best understood as an audit layer:

- TestU01 and PractRand remain necessary distributional evidence;
- structural dispersion measures seed-conditioned Monte Carlo stability;
- stream hashes and manifests support backend reproducibility;
- no empirical score replaces a mathematical theorem.

9.1 Why Seed Sensitivity Matters

In many Monte Carlo workflows, changing the seed is treated as a harmless way to obtain a fresh pseudo-random stream. This is reasonable only if the generator and the estimator behave stably over the tested seed set. If variance differs substantially from one seed to another, then a single seed can produce a misleading impression of estimator quality.

The diagnostic ratio $\rho_f(B)$ is intentionally simple. It does not attempt to summarize every possible defect. It asks whether the empirical variance of a declared functional changes materially across a declared seed set. Because the diagnostic is conditional on B, N, R, f, and the backend, it should always be reported with these parameters.

9.2 Why Backend Reproducibility Matters

Modern simulations increasingly mix CPU and GPU implementations. If a generator produces backend-dependent streams, it becomes difficult to distinguish numerical effects from random-stream effects. Bit-exact reproducibility is not always required for every application, but when it is claimed, it should be checked through raw stream hashes rather than inferred from statistical battery pass/fail results.

The imported BigCrush logs are useful because they show that the tested backend streams passed the same battery. They do not, by themselves, prove that the streams are identical. That stronger claim requires fixed-seed output hashes or a replay harness.

10 Limitations

The current public material has important limitations:

- `Montmory_CTACM` is proprietary and undisclosed;
- raw SmallCrush logs are not yet imported;
- raw x86 CPU logs are not yet imported;
- stream adapter source or hash is not yet imported;
- backend implementation hashes and build commands are not yet imported;
- bit-exact stream hashes are not yet imported;
- full per-seed structural variance tables are not yet imported;
- throughput benchmarks against Philox4x32-10 are not yet included.

Consequently, the current note should be read as a public methodological and empirical report, not as a complete independent reproducibility package.

11 Recommended Public Release Path

The most useful next public release would separate three layers.

Layer 1: paper artifact. A stable PDF describing the diagnostic framework, the public formulas, the current BigCrush results, and the reported structural-dispersion table. This is the role of the present document.

Layer 2: reproducibility bundle. A repository snapshot containing raw logs, manifests, checksums, stream adapter source or hashes, command lines, and environment metadata. This layer should allow another user to verify that the reported artifacts match the published hashes.

Layer 3: optional open implementation. A non-proprietary reference implementation of the diagnostic harness over open generators. This layer does not require disclosure of `Montmory_CTACM`. It would make the framework itself reviewable and reusable.

This release path avoids overclaiming while still making the scientific contribution visible: the proposed structural diagnostic methodology is public, even if the controlled test vehicle remains proprietary.

12 Conclusion

This note introduces a structural diagnostic framework for PRNG evaluation in Monte Carlo workloads. The framework formalizes seed sensitivity, functional variance dispersion, open structural scoring, transition instability, and cross-backend reproducibility checks.

The imported BigCrush logs provide computational evidence that `Montmory_CTACM` passed BigCrush with 160/160 statistics on CPU ARM, Metal GPU, and CUDA GPU for seed 42. The reported structural dispersion tables indicate seed-dependent variance amplification on practical integrands, especially rare-event indicators, for several tested PRNG families.

The main conclusion is not a ranking of generators. It is that PRNG evaluation should treat seed sensitivity and functional stability as first-class empirical properties, complementary to classical statistical batteries.

This matters for large-scale simulations in finance, physics, engineering, and machine learning, where many runs are compared across seeds, devices, and implementation backends. In such settings, reproducible stream hashes and functional-stability diagnostics can help distinguish genuine model behaviour from artifacts introduced by seed choice or backend-specific random streams.

Acknowledgement of Artifact Status

The accompanying Git repository contains raw BigCrush logs and a manifest with SHA-256 hashes. It does not contain confidential correspondence, proprietary recurrence details, or implementation code for the proprietary generator. Public claims should be limited to the artifacts and definitions included in the repository.

A Imported TestU01 Manifest

Artifact	Backend	Suite	SHA-256
CPU ARM log	CPU ARM, Darwin	BigCrush	`85696c1d4908da2e225083065c0f44a5cc04438c05d193823f8aec62f7a3feb2`
Metal log	Metal GPU, Darwin	BigCrush	`e19a9d95020b8a8bb9b84fe64128f87393cdae263332edb7df2507250b13088e`
CUDA log	CUDA GPU, Linux	BigCrush	`f88b13037aee7c97bd38bc55d1b650524d7afdbe7585ab96de7da5452aecf312`

The corresponding repository filenames are:

```
Math/PRNG/tests/testu01/raw/montmory_ctacm_cpu_arm_bigcrush_seed42.log
Math/PRNG/tests/testu01/raw/montmory_ctacm_metal_bigcrush_seed42.log
Math/PRNG/tests/testu01/raw/montmory_ctacm_cuda_bigcrush_seed42.log
```

B Reproducibility Checklist

To promote the current material from computational evidence to a stronger reproducibility package, the following artifacts should be added:

- raw SmallCrush logs for every claimed backend;
- x86 CPU raw logs if x86 remains part of the backend claim;
- TestU01 stream adapter source or source hash;
- backend implementation commit hashes;
- compiler versions, flags, hardware details, and exact commands;
- fixed-seed stream hashes for bit-exact backend comparison;
- complete per-seed variance tables;
- a replay script validating expected log hashes and stream hashes.

References

[1] P. L'Ecuyer and R. Simard. TestU01: A C library for empirical testing of random number generators. *ACM Transactions on Mathematical Software*, 33(4), 2007.

[2] C. Doty-Humphrey. PractRand random number test suite. Software test suite and documentation, versioned public releases.

[3] M. Matsumoto and T. Nishimura. Mersenne Twister: A 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, 8(1), 1998.

[4] M. E. O'Neill. PCG: A family of simple fast space-efficient statistically good algorithms for random number generation. Technical report HMC-CS-2014-0905, Harvey Mudd College, 2014.

[5] S. Vigna. Further scramblings of Marsaglia's xorshift generators. *Journal of Computational and Applied Mathematics*, 315:175–181, 2017.

[6] J. K. Salmon, M. A. Moraes, R. O. Dror, and D. E. Shaw. Parallel random numbers: As easy as 1, 2, 3. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC11)*, 2011.

[7] P. L'Ecuyer. Good parameter sets for combined multiple recursive random number generators. *Operations Research*, 47(1):159–164, 1999.